

Lab 6: Linear Regression Models (Continued)

This lab will be an extension of what we did last week, adding on our skills of creating linear regression models, based on some of the ideas you have been learning during lecture.

Specifically, this lab will focus on:

- Removing outliers from a dataset
- Normalizing and standardizing a dataset
- Creating interaction terms
- Analyzing and plotting model results
- Build on skills we learned last week

Note that this lab relies heavily on concepts learned during Lab 6 (intro to regressions) and Lab 4 (plotting). Be sure to look back at those labs if you are having any trouble with this one.

As usual, we begin by importing packages. All of these are packages we have learned in the past.

```
#Import packages
import numpy as np
import pandas as pd
import datetime as dt
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

#Additional command to tell jupyter notebook to show the plots
directly underneath the code cells
%matplotlib inline
```

Next, we import our data, look at the summary info and metadata, and change the appropriate column to a datetime format.

```
#Import csv file with data
e_df = pd.read_csv('JES Energy_Lab7Tutorial.csv')

#Inspect variables
print(e_df.head(5))

e_df.info()

#Describe data to see if there are any NaN values.
#We can see from the info and describe outputs there are no NaN
values.
e_df.describe()
```

	allDate	avgTemp	eU	MonMed	classSession	dayType
season						
0 1/14/2009 12:00		47.0	11924	13699.0	0	wk
Winter						
1 1/15/2009 12:00		46.0	12006	13699.0	0	wk
Winter						
2 1/16/2009 12:00		41.5	12009	13699.0	0	wk
Winter						
3 1/17/2009 12:00		57.5	10881	13699.0	0	wknd
Winter						
4 1/18/2009 12:00		61.0	11238	13699.0	0	wknd
Winter						

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 3093 entries, 0 to 3092
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	allDate	3093 non-null	object
1	avgTemp	3093 non-null	float64
2	eU	3093 non-null	int64
3	MonMed	3093 non-null	float64
4	classSession	3093 non-null	int64
5	dayType	3093 non-null	object
6	season	3093 non-null	object

```
dtypes: float64(2), int64(2), object(3)
```

```
memory usage: 169.3+ KB
```

	avgTemp	eU	MonMed	classSession
count	3093.000000	3093.000000	3093.000000	3093.000000
mean	70.559328	12037.070158	12384.691077	0.604914
std	14.474577	2330.454011	1833.150927	0.488948
min	21.500000	0.000000	8286.000000	0.000000
25%	60.000000	10416.000000	10862.000000	0.000000
50%	72.500000	12098.000000	12441.000000	1.000000
75%	83.000000	13600.000000	13679.000000	1.000000
max	96.000000	30875.000000	16276.000000	1.000000

```
#Don't forget to turn the necessary column into a datetime!
```

```
e_df['allDate'] = pd.to_datetime(e_df['allDate'])
```

```
#Print info again to confirm conversion to datetime
```

```
e_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 3093 entries, 0 to 3092
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	allDate	3093 non-null	datetime64[ns]
1	avgTemp	3093 non-null	float64

```
2    eU          3093 non-null    int64
3    MonMed      3093 non-null    float64
4    classSession 3093 non-null    int64
5    dayType     3093 non-null    object
6    season      3093 non-null    object
dtypes: datetime64[ns](1), float64(2), int64(2), object(2)
memory usage: 169.3+ KB
```

Exploratory Plotting

Whenever you get a dataset, the first thing you will want to do is make sure the dataset is "clean," and that means without errors or outliers. An easy way to do that is to make a quick plot to look at the data. It doesn't have to be pretty- just to take a quick peek at the data.

```
# Make a plot of the EU column to see what we're working with.
# Doesn't have to be pretty - just take a quick look at the data

#Create a figure
f = plt.figure()

#Make a quick plot of the energy consumption
plt.plot(e_df['allDate'],e_df['eU'])

[<matplotlib.lines.Line2D at 0x7fe44c101290>]
```



Immediately we see there are some outliers to take care of!

There are many ways to test and remove outliers. Examples include statistical tests, moving filters, distance away from the IQR, and others. Here, we decide that if any specific data point is

more than 3 standard deviations above or below the mean of the data, then it is considered an outlier and removed from the dataset.

To do this, we need to calculate the mean and standard deviation of the energy consumption, and then remove any values that are outside the acceptable range.

```
#Let's remove some outliers.

#Calculate the standard deviation
eu_std = np.std(e_df['eU'],ddof=1)

#Calculate the mean
eu_mean = np.mean(e_df['eU'])

#Determine what our upper threshold is
upper_threshold=eu_mean+3*eu_std

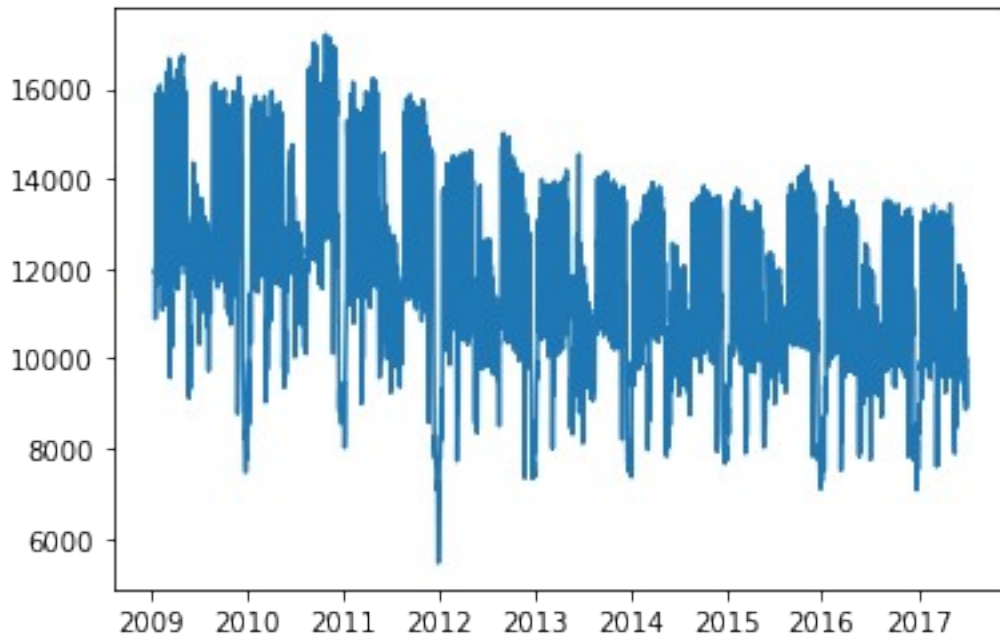
#Determine what our lower threshold is
lower_threshold=eu_mean-3*eu_std

#Only keep rows of the dataframe where the value
#in the eU column is below the upper threshold AND above the lower
threshold
e_df = e_df.loc[(e_df['eU']<upper_threshold) &
(e_df['eU']>lower_threshold),:]

#Make a plot just to see how it turned out
f = plt.figure()
plt.plot(e_df['allDate'],e_df['eU'])

#Let's also take a look at the info again.
#What do you notice compared to before?
#Can you tell how many points got removed?
e_df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 3083 entries, 0 to 3092
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   allDate         3083 non-null   datetime64[ns]
1   avgTemp         3083 non-null   float64
2   eU              3083 non-null   int64
3   MonMed          3083 non-null   float64
4   classSession    3083 non-null   int64
5   dayType         3083 non-null   object
6   season          3083 non-null   object
dtypes: datetime64[ns](1), float64(2), int64(2), object(2)
memory usage: 192.7+ KB
```



Next, we add an interaction term. Here, we are adding a column which contains the value of the normalized monthly median multiplied by the normalized average temperature.

```
#Let's add another column which is the product of temperature and monthly median
e_df['Temp_x_Med'] = e_df['avgTemp'] * e_df['MonMed']
```

Next, we want to normalize (or also standardize) our numerical data so that all of the values are in a similar range. Refer to the lecture notes for more information regarding normalization and standardization.

Here, we normalize each of the 3 columns which contain numerical data: average temperature, daily energy consumption, and monthly median consumption. For each of those columns, we need to know the minimum and maximum, and then we can convert the values to the normalized values.

```
#The columns of numerical variables are avg temp, eU, and MonMED
#Normalize EU, MonMED, avgTemp, and Temp_x_Med

#Energy Usage
#min of eU
eU_min = np.min(e_df['eU'])

#max of eU
eU_max = np.max(e_df['eU'])

#normalize eU column
e_df['eU'] = (e_df['eU'] - eU_min) / (eU_max - eU_min)
```

```

#MonMed
#min of MonMed
MonMed_min=np.min(e_df['MonMed'])

#max of MonMed
MonMed_max=np.max(e_df['MonMed'])

#normalize MonMed column
e_df['MonMed']=(e_df['MonMed']-MonMed_min)/(MonMed_max-MonMed_min)

#AvgTemp
#avgTemp minimum
avgTemp_min=np.min(e_df['avgTemp'])

#avgTemp maximum
avgTemp_max=np.max(e_df['avgTemp'])

#Normalize
e_df['avgTemp']=(e_df['avgTemp']-avgTemp_min)/(avgTemp_max-
avgTemp_min)

#Temp_x_Med
#avgTemp minimum
Temp_x_Med_min=np.min(e_df['Temp_x_Med'])

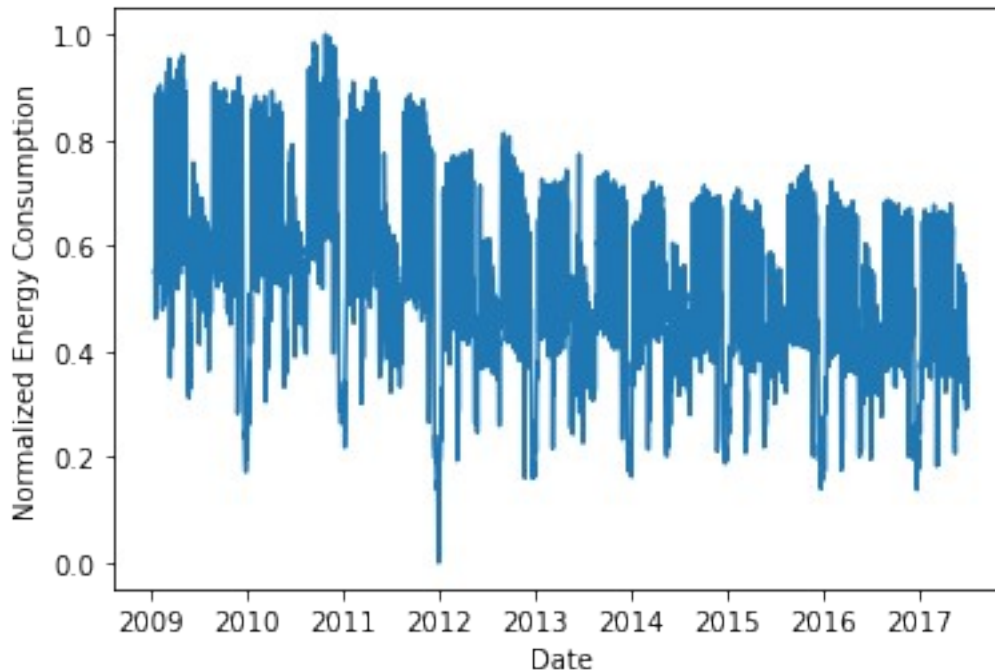
#avgTemp maximum
Temp_x_Med_max=np.max(e_df['Temp_x_Med'])

#Normalize
e_df['Temp_x_Med']=(e_df['Temp_x_Med']-Temp_x_Med_min)/(Temp_x_Med_max-
Temp_x_Med_min)

#Make another plot of eU just to see how it turned out. You can plot
other columns as well

f2=plt.figure()
plt.plot(e_df['allDate'],e_df['eU'])
plt.xlabel('Date')
plt.ylabel('Normalized Energy Consumption')
Text(0, 0.5, 'Normalized Energy Consumption')

```



In the plot above, you can tell that the normalization worked because all of the values are between 0 and 1, but the overall shape of the data is the same.

Linear Regression Model

We're ready to get modeling! Yahoo!

But before we do, we must create dummy variables from the variables which are categorical. Last time, we learned the one-hot-encoder function. Here, we use a different function from the pandas library which performs the same task but is more simple.

See link for how the `get_dummies` function works:

https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html

Note that we are dropping one of the columns for each of the categorical variables! Refer to Lecture 11 for an explanation about this.

```
#Remember we have the categorical terms so we need to create dummy variables
#Last week we used one-hot-encoder.
#This week we're using a different function that does something similar, get_dummies
```

```
e_df_dummy = pd.get_dummies(e_df, columns=['classSession',
'dayType', 'season'], drop_first=True)
```

```
e_df_dummy.head(5)
```

	allDate	avgTemp	eU	MonMed	Temp_x_Med	\
0	2009-01-14 12:00:00	0.342282	0.550510	0.677472	0.321761	

1	2009-01-15	12:00:00	0.328859	0.557483	0.677472	0.310022
2	2009-01-16	12:00:00	0.268456	0.557738	0.677472	0.257195
3	2009-01-17	12:00:00	0.483221	0.461820	0.677472	0.445024
4	2009-01-18	12:00:00	0.530201	0.492177	0.677472	0.486112

	classSession_1	dayType_wknd	season_Spring	season_Summer	season_Winter
0	0	0	0	0	0
1					
1	0	0	0	0	0
1					
2	0	0	0	0	0
1					
3	0	1	0	0	0
1					
4	0	1	0	0	0
1					

Next, we want to separate out output variables from our input variables for the model. To create the y variable, we are only taking the eU column. To get the predictor variables, we only dropping the eU column.

```
#Create 2 seperate dataframes of the input variables and output variable
```

```
y = e_df_dummy['eU']
```

```
x = e_df_dummy.drop(['eU'],axis=1)
```

This part should look familiar to last week! Making a linear regression model and then evaluating the results for the training dataset.

However, there is just one extra step. In the next cell (as you will see), we want to know which dates are associated with the testing and training datasets. That's why we do the test/train split, THEN drop the date column from the x data. The date itself is not a predictor, but we will make a plot later and we will want the associated dates for the training and testing datasets. If we were not making this plot, then we could have dropped the date column in the previous step.

```
#Split data between training and testing sets
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3,random_state=42)
```

```
#Extract the dates for the testing and training datasets
```

```
x_train_date = x_train['allDate']
```

```
x_test_date = x_test['allDate']
```

```
#Drop the date columns because they are not predictors
```

```
x_train=x_train.drop(['allDate'],axis=1)
```



```

x_test=x_test.drop(['allDate'],axis=1)

#Create linear model
lm=LinearRegression()

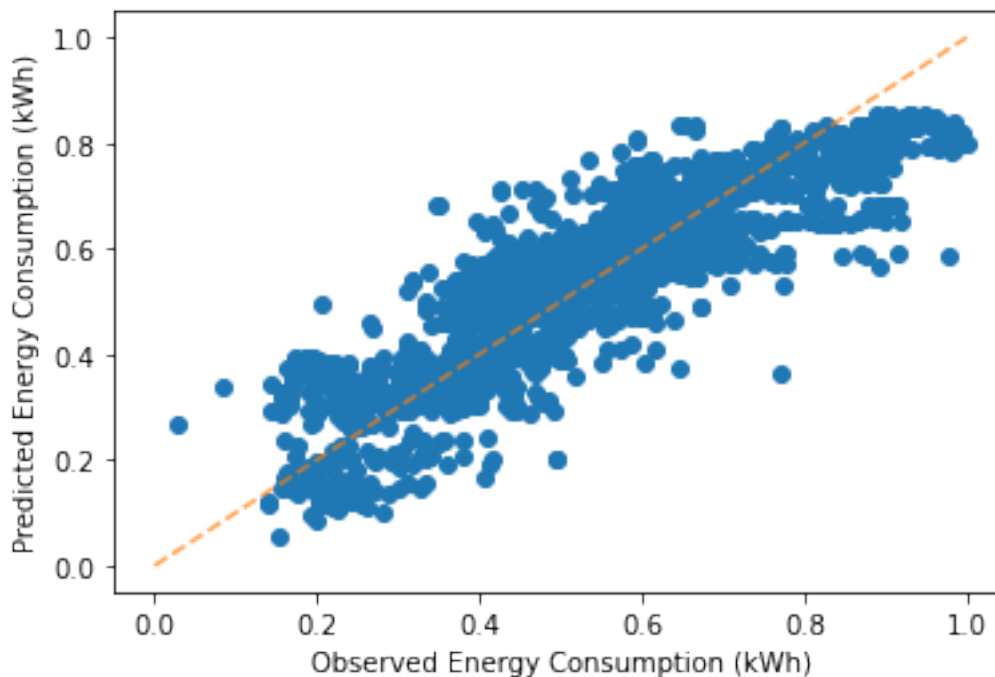
#Fit linear model
lm.fit(x_train,y_train)

#Use model to predict consumption values based on the training values
y_train_predicted=lm.predict(x_train)

#Plot results (this should ring a bell from Lab 6)
f3=plt.figure()

plt.plot(y_train,y_train_predicted,ls='',marker='o')
plt.plot(np.array([0,1]),np.array([0,1]),ls='--',alpha=.7)
plt.xlabel('Observed Energy Consumption (kWh)')
plt.ylabel('Predicted Energy Consumption (kWh)')
Text(0, 0.5, 'Predicted Energy Consumption (kWh)')

```



Create Time Series Plot

We're almost there! Now, as an additional step, we want to plot the predicted and actual consumption values against the date so we can see how the values compare on a timeseries. This plot is like what we saw from Ohm Hour (see lab slides). We can use a plot like this to try to see when would be the best times to encourage energy savings!

Note that in the previous step, the dates were separated AFTER the test_train split for the purpose of making this plot. We needed the dates for the training data to be on the x axis of the plot.

```
#Create the figure
f4=plt.figure()
#Plot results
plt.plot(x_train_date,y_train,ls='',marker='o')
plt.plot(x_train_date,y_train_predicted,ls='',marker='o')

#Adjust x axis to be optimized for datetimes (See lab 4 for more info)
plt.gcf().autofmt_xdate()

#So that we can better understand the plot
#let's just look at a couple months of data
#Adjust the extent of the x axis so that it
#only shows from January 1, 2014 to March 1, 2014
plt.xlim([dt.date(2014, 1, 1), dt.date(2014, 3, 1)])

#Add labels
plt.xlabel('Date')
plt.ylabel('Normalized Energy Consumption')
plt.legend(['Actual Consumption', 'Predicted Consumption'])

<matplotlib.legend.Legend at 0x7fe44cac4f90>
```

